

LC 787: Cheapest Flights Within K Stops

Authored by Arpit Raj, LNMIIT Jaipur | July 24, 2026

Problem Overview

Companies: De Shaw, MMT, Stripe

- `from`, `to`, and `price` are given.
- `src` and `dst` are given.
- **Goal:** Need to reach from `src` to `dst` in minimum price while ensuring at most `k` stops (vertices) are stumbled.

Strategic Approaches

Approach 1: State BFS

Time Complexity: $O(E + K)$

Approach 2: Dijkstra's Algorithm Variant

Time Complexity: $O(E \log(VK))$

Key Insights:

- However, normal Dijkstra's only stores `dist[node]` . It doesn't know how many stops were taken.
- So, we store `dist[node][stops]` .
- The Priority Queue (PQ) becomes: `{cost, {node, stops}}` .
- `nextcost = currcost + edgeweight`
- `nextstop = currstop + 1`

Example Mapping:

Imagine a path: `0 → 1 → 2 → 3`

This path has **stops = 2** and **edges = 3**.

Hence, in vector initialization we will use `(k + 2)` and not `(k + 1)` .

Implementations

Algorithm using Priority Queue (Dijkstra Variant)

```
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        vector<vector<pair<int,int>>> adj(n);
        for(auto &it:flights){
            int u=it[0];
            int v=it[1];
            int c=it[2];

            adj[u].push_back({v,c});
        }

        // we will store {cost,{node,stops}};
        // and dist[node][stop] to store mincost;
        // stop is needed along with dist because let us say in future
        // instead of is this the cheapest way to reach nextnode
        // ask is this the cheapest way to reach nextnode with this number of stops
        priority_queue<pair<int,pair<int,int>>,vector<pair<int,pair<int,int>>>,greater<pair<int,pair<int,int>>>>
        vector<vector<int>> dist(n,vector<int>((k+1),INT_MAX)); // 0 is also a number of stops

        pq.push({0,{src,0}}); // intially no stop
        dist[src][0]=0;
```

```

while(!pq.empty()){
    int currnode=pq.top().second.first;
    int currdiss=pq.top().first;
    int currstops=pq.top().second.second;
    pq.pop();

    if(currdiss>dist[currnode][currstops]){
        continue;
    }

    if(currnode==dst) return currdiss;

    for(auto &it:adj[currnode]){
        int newnode=it.first;
        int cost=it.second;

        int newdiss=currdiss+cost;
        int newstop;
        if(newnode==dst) newstop=currstops; // destination is not counted as a stop
        else newstop=currstops+1;

        if(newstop<=k){
            if(newdiss<dist[newnode][newstop]){
                dist[newnode][newstop]=newdiss;
                pq.push({newdiss,{newnode,newstop}});
            }
        }
    }
}
return -1;
};

```

Algorithm using Queue (BFS Level-by-Level)

```

// APPROACH 2
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        vector<vector<pair<int,int>>> adj(n);
        for(auto &it:flights){
            int u=it[0];
            int v=it[1];
            int c=it[2];
            adj[u].push_back({v,c});
        }

        // step 2 prepare queue to store {cost,node}

```

```

// we DO NOT store stops in queue
// because BFS level itself tells us how many stops we have used
queue<pair<int,int>> q;
vector<int> dist(n,INT_MAX);

q.push({0,src});
dist[src]=0;

int currstops=0; // initially no stop has been used

while(!q.empty() && currstops<=k){

    int currsz=q.size();

    // maan lo abhi queue me wahi stops he jo 5 station ke baad aaye he
    // to in sabhi ka next wo station denge jo 6 station baad aaye he
    while(currsz--){
        int currdiss=q.front().first;
        int currnode=q.front().second;
        q.pop();

        if(currdiss>dist[currnode]) continue;

        for(auto &it:adj[currnode]){
            int newnode=it.first;
            int cost=it.second;
            int newdiss=currdiss+cost;

            if(newdiss<dist[newnode]){
                dist[newnode]=newdiss;
                q.push({newdiss,newnode});
            }
        }
        // next BFS level means one more stop has been used
        currstops++;
    }

    if(dist[dst]==INT_MAX) return -1;
    return dist[dst];
}
};

```